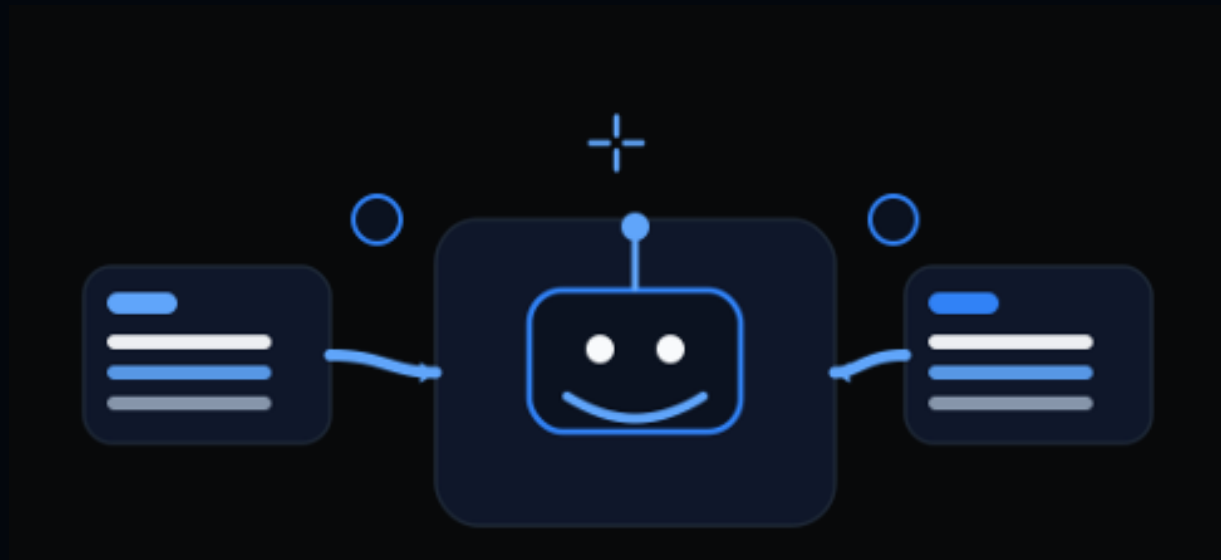


유스AI프로젝트 P.I.N.E 2기



우리가 여기에 모인 이유



오늘의 핵심 문장

작은 기능 하나를 직접 만들어본다.

타인이 작성한 코드를 활용해 작은 기능 하나를 만들어본다.

타인이 작성한 코드를 응용해 여러 기능으로 확장해본다.

출력

입력

조건문

함수

이름!!!

지금 이 순간 여러분이 느낀 감정!!

지금 꼭 하고 싶은 말!!

짧게 한 문장으로 말하기

첫 번째 경험

화면에 글자 출력하기

내가 쓴 글자가 화면에 실제로 나타나는 첫 경험이 중요합니다.

```
1 print("안녕하세요")
2 print("여러분의 시작을 응원합니다.")
3 print("My name is Jiyong")
```

print는 화면에 보여달라는 뜻이고, 따옴표 안에는 보여줄 글자를 넣습니다.

print 화면에
보여줘

() 시킬 내용을
넣는 자리

"글자" 문자
데이터

import

이미 만들어진 도구를 불러오는 것

모든 기능을 처음부터 만들 필요는 없습니다.

다른 사람이 만들어 둔 기능을 불러와 바로 사용할 수 있습니다.

```
1 import random
2
3 print(random.randint(1, 3))
4 print(random.randint(10, 99))
```

`random.randint(a, b)`는 a부터 b 사이의 정수 하나를 무작위로 뽑습니다.

import 가져와라

random 랜덤 도구
이름

. 안에 있는 기능
꺼내기

randint(1, 3) 1부터 3 사이
정수 뽑기

파이썬이 몰래 가지고 있는 기능들?!

내장 함수

파이썬이 이미 여러분을 위해 만든, 바로 꺼내 쓸 수 있는 기능/도구들이에요

```
1 print()  
2 int()  
3 input()  
4 str()  
5 len()  
...
```

import

랜덤 숫자에 의미 붙이기

숫자를 그냥 뽑는 것보다, 예시를 붙이면 기능처럼 느껴 집니다.

같은 문법으로 다른 서비스 아이디어를 금방 만들 수 있습니다.

```
1 import random
2
3 money = random.randint(1000, 50000)
4 print("오늘 받을 용돈:", money)
5
6 score = random.randint(1, 100)
7 print("오늘의 퀴즈 점수:", score)
8
9 luck = random.randint(1, 5)
10 print("오늘의 운세 점수는:", luck)
```

변수에 저장한 뒤 문장과 함께 출력하면 더 실제 서비스처럼 보입니다.

다른 모듈도 불러오기

random 말고 다른 모듈도 직접 불러와 결과를 출력해봅시다.

해보기

- time, math, datetime 중 하나를 골라 import 해보기
- print()로 결과가 화면에 보이게 만들기
- 팀원과 서로 다른 모듈 하나씩 시도해보기

완료 기준

- import 문 1개
- 화면에 출력되는 결과 1개
- 무엇을 출력했는지 한 문장으로 설명하기

```
1 import math
2 print(math.sqrt(81))
3
4 from datetime import datetime
5 print(datetime.now())
```

출력 결과가 보이면 성공입니다. 값의 의미를 한 문장으로 설명해 보세요.

time

math

datetime

변수는 무엇인가

값을 담아두는 이름표

같은 값을 여러 번 다시 쓰기 위해 이름표를 붙여 저장합니다.

문장을 바꾸고 싶을 때도 변수 하나만 고치면 됩니다.

```
1 name = "민수"  
2 bank = "KB"  
3 goal = 50000  
4  
5 print(name)  
6 print(bank)  
7 print(goal)
```

= 는 오른쪽 값을 왼쪽 이름에 저장한다는 뜻입니다.

= 오른쪽 값을
왼쪽에 저장

bank 내가
정한 이름표

"KB" 글자
데이터

변수는 왜 필요할까

같은 말을 여러 번 쓸 때 편해집니다.

같은 문장을 반복할수록 수정이 번거로워집니다.

변수를 쓰면 이름만 바뀌어도 여러 줄이 한 번에 같이 바뀝니다.

```
1 print("오늘은 저축하는 날")
2 print("오늘은 저축하는 날")
3 print("오늘은 저축하는 날")
4
5 message = "오늘은 저축하는 날"
6 print(message)
7 print(message)
8 print(message)
```

반복되는 값은 변수로 묶어두면 관리가 쉬워집니다.

반복 줄이기

수정 편함

재사용

파이썬의 대표 값 3가지

글자 · 숫자 · 참/거짓

처음에는 이 세 가지만 확실히 잡아도 충분합니다.

문장, 수량, 상태를 구분해서 다룰 수 있으면 작은 프로그램이 만들어집니다.

```
1 name = "Jiyong"
2 weight = 70
3 is_student = False
4 like_apple = True
5
6 print(name)
7 print(weight)
```

문자는 따옴표, 숫자는 그대로, 참/거짓은 True 또는 False로 씁니다.

"Jiyong" 문자

70 숫자

True/False 참 / 거짓

입력이란 무엇인가

컴퓨터가 사람에게 질문하는 것

이 순간부터 프로그램은 혼자 말하는 것이 아니라 사람과 대화하기 시작합니다.

```
1 name = input("이름을 입력하세요: ")
2 food = input("좋아하는 음식은? ")
3
4 print(name)
5 print(food)
```

input() 안의 문장은 사용자에게 보이는 질문입니다.

input 입력 받기

"질문" 화면에
보여줄 말

name 입력값이
저장될 이름표

입력과 출력을 연결하기

사람의 입력이 결과를 바꾸기 시작합니다.

먼저 묻고, 받은 답을 다시 결과에 활용해보세요.
이제부터 프로그램은 사용자마다 다른 결과를 보여줄 수 있습니다.

```
1 name = input("이름을 입력하세요: ")
2 print("안녕하세요,", name)
3
4 branch = input("자주 가는 은행은? ")
5 print(branch, "을 자주 이용하는군요.")
6
7 product = input("관심 있는 상품은? ")
8 print(product, "설명을 준비해볼게요.")
```

입력값을 변수에 저장한 뒤 print에서 다시 사용하면 됩니다.

질문

대답

맞춤 결과

True를 활용하는 방법

만약~라면: if

조건식을 검사하고 맞으면 아래 행동을 실행합니다.
점수, 나이, 금액처럼 비교할 수 있는 값에서 많이 씁니다.

```
1 score = 85
2 if score >= 80:
3     print("좋아요")
4
5 money = 5000
6 if money >= 3000:
7     print("간식을 살 수 있어요")
8
9 save = 20000
10 if save >= 10000:
11     print("저축 성공")
```

if 다음 줄부터는 들여쓰기로 같은 묶음이라는 것을 보여줍니다.

if 만약

>= 이상이라면

: 조건 설명 끝,
아래 행동 시작

들여쓰기 조건이
맞을 때 실행할 행동

False 활용법

그게 아니라면: else

if가 맞지 않을 때 실행할 다른 길을 만듭니다.
둘 중 하나의 결과를 보여줄 때 매우 자주 쓰입니다.

```
1 score = 60
2 if score >= 80:
3     print("통과")
4 else:
5     print("조금 더 공부")
6
7 spend = 12000
8 if spend <= 10000:
9     print("예산 지킴")
10 else:
11     print("지출 점검 필요")
```

if와 else는 한 쌍으로 움직입니다. 조건이 맞지 않으면 else가 실행됩니다.

else

두 갈래

결과 분기

if-else 3개 만들어보기

그중 1개는 숫자를 입력받아 비교해보세요.

조건

- 1개는 점수 기준
- 1개는 금액 기준
- 1개는 자유 주제

권장 흐름

- 1) input으로 값을 받기
- 2) 숫자가 필요하면 int()로 바꾸기
- 3) if-else로 다른 결과 출력하기

완료 기준

- 조건문 3개
- 그중 1개는 직접 입력받기

```
1 age = int(input("나이를 입력하세요: "))
2
3 if age >= 20:
4     print("성인입니다")
5 else:
6     print("미성년자입니다")
```

input으로 받은 값은 기본적으로 문자입니다. 숫자 비교를 할 때는 int()가 자주 필요합니다.

if-else

input

int()

내장 함수 range

차례대로 숫자를 만들어요

0, 1, 2 까지를 차례로 자동으로 만들 수 있을까요?

```
1 range(3)
2 1 in range(0, 3)
```

단순 반복 피하는 방법 for + range

같은 일을 자동으로 여러 번 하기

사람이 똑같은 코드를 여러 번 쓰는 대신, 반복 문장으로 자동화합니다.

```
1 for i in range(3):  
2     print(i)
```

for 반복 시작

i 반복할 때 잠깐 쓰는 이름표

range(3) 3번 반복

반복문으로 같은 일 줄이기

같은 일을 자동으로 여러 번 하기

사람이 같은 코드를 여러 번 쓰는 대신, 반복 문장으로 자동화할 수 있습니다.

```
1 for i in range(3):  
2     print("안녕하세요")  
3  
4 for i in range(5):  
5     print("금융 퀴즈 시작!")  
6  
7 for i in range(2):  
8     print("오늘도 저축 체크")
```

range(3)은 3번 반복하라는 뜻입니다.

for 반복 시작

i 반복할 때 잠깐
쓰는 이름표

range(3) 3번
반복

리스트 []

여러 개를 한 번에 담는 상자

하나만 담고 싶으면 변수, 여러 개를 모아두고 싶으면 리스트를 씁니다.

한 묶음으로 관리할 수 있어서 반복문과 특히 잘 어울립니다.

```
1 foods = ["김밥", "라면", "떡볶이"]
2 banks = ["KB", "신한", "우리"]
3 products = ["예금", "적금", "카드"]
4
5 print(banks)
6 print(products)
```

[]는 리스트를 만드는 표시이고, 쉼표로 값을 구분합니다.

리스트

[]

여러 값

리스트에서 값 꺼내보기

리스트의 항목 하나 꺼내기

첫 번째 값은 [0]부터 시작합니다.

번호를 지정하면 원하는 항목 하나만 꺼낼 수 있습니다.

```
1 foods = ["김밥", "라면", "떡볶이"]
2 banks = ["KB", "신한", "우리"]
3 products = ["예금", "적금", "카드"]
4
5 print(foods[0])
6 print(banks[1])
7 print(products[2])
```

0은 첫 번째, 1은 두 번째, 2는 세 번째를 뜻합니다.

자동화의 시작

리스트 + for

리스트 안의 값을 하나씩 꺼내서 같은 작업을 할 수 있습니다.

실제로는 추천 문구 만들기, 안내 메시지 반복 출력에 자주 씁니다.

```
1 banks = ["KB", "신한", "우리"]
2 for bank in banks:
3     print(bank)
4
5 products = ["예금", "적금", "카드"]
6 for product in products:
7     print(product, "추천")
```

for 뒤의 bank, product는 반복할 때 잠깐 쓰는 이름표입니다.

리스트+for

자동화

추천 출력

def로 함수 이름 붙이기

자주 쓸 동작을 저장해두기

같은 코드를 여러 번 쓰는 대신, 자주 쓸 동작에 이름을 붙여 저장할 수 있습니다.

```
1 def greet():
2     print("안녕하세요")
3
4 def hello():
5     print("반갑습니다")
6
7 def quiz_start():
8     print("금융 퀴즈를 시작합니다")
9
10 hello()
11 quiz_start()
```

def는 함수를 만든다는 뜻이고, 마지막 줄처럼 함수 이름()을 써야 실행됩니다.

def
함수를 만든다

hello 함수
이름

() 입력이
들어갈 자리

들여쓰기 함수에
포함된 내용

입력이 있는 함수

넣는 값에 따라 결과가 달라집니다.

함수 안에 넣는 값이 달라지면 결과도 달라집니다.

같은 기능을 여러 상황에 맞춰 재사용하기 좋습니다.

```
1 def greet(name):  
2     print("안녕하세요,", name)  
3  
4 def welcome(branch):  
5     print(branch, "지점에 오신 것을 환영합니  
다")  
6  
7 def recommend(product):  
8     print(product, "설명을 시작할게요")  
9  
10 greet("지민")  
11 recommend("예금")
```

괄호 안의 name, branch, product는 함수에 넣는 값입니다.

매개 변수

입력 값

결과 변화

함수는 왜 필요할까요

반복되는 코드를 줄여줍니다.

같은 행동을 여러 번 해야 할 때 특히 유용합니다.

기능 별로 나뉘어 있어 나중에 다시 읽기도 쉬워집니다.

```
1 def greet(name):  
2     print("안녕하세요, ", name)  
3  
4 greet("민수")  
5 greet("지민")  
6 greet("수아")  
7  
8 def notice(product):  
9     print(product, "상품 설명을 열었습니다")  
10  
11 notice("적금")
```

비슷한 문장을 여러 번 쓰지 않고, 함수 한 개를 여러 번 호출하면 됩니다.

코드 줄이기

재사용

가독성

함수 3개 만들어보기

가장 잘 만든 함수 하나는 꼭 발표해봅시다.

만들어볼 것

- 1개: 입력 없이 실행되는 함수
- 1개: 이름을 넣으면 문장이 바뀌는 함수
- 1개: 금융 주제로 자유롭게 만드는 함수

예시 주제

- 저축 응원 메시지
- 오늘의 금융 퀴즈 시작
- 상품 추천 한 줄

발표

- 가장 마음에 드는 함수 1개를 골라 설명하기

```
1 def cheer(name):  
2     print(name, "님, 오늘도 저축 도전!")  
3  
4 def product_intro(product):  
5     print(product, "기본 설명을 보여줍니다")
```

함수 이름은 기능이 보이게 짓는 것이 좋습니다.

딕셔너리 {}

이름표와 값이 짝지어진 상자

리스트가 순서대로 담는 상자라면, 딕셔너리는 이름표와 값을 짝지어 담는 상자입니다.

```
1 student = {  
2     "name": "지민",  
3     "grade": 1  
4 }  
5  
6 account = {  
7     "bank": "KB",  
8     "money": 5000  
9 }  
10  
11 print(account)
```

"bank"처럼 항목 이름을 붙여두면 어떤 값인지 읽기 쉽습니다.

"bank" 항목
이름

5000 그
항목의 값

{ } 딕셔너리
표시

딕셔너리 {}

이름표와 값이 짝지어진 상자

```
1 student = {  
2     "name": "지민",  
3     "grade": 1  
4 }  
5  
6 print(student["name"])  
7  
8  
9
```

"name" 항목 이름

"지민" 그 항목의 값

{ } 딕셔너리 표시

딕셔너리로 간단한 응답 만들기

아주 쉬운 챗봇 느낌 만들기

입력한 단어에 맞는 문장을 미리 준비해두고 꺼내 쓰는 방식입니다.

간단한 FAQ, 상품 한 줄 설명, 분류기 느낌을 만들 수 있습니다.

```
1 tips = {  
2     "예금": "안정적으로 돈을 모을 때 좋아요.",  
3     "적금": "매달 꾸준히 모을 때 좋아요.",  
4     "카드": "소비 관리가 중요해요."  
5 }  
6  
7 product = input("궁금한 금융 상품은? ")  
8 print(tips.get(product, "아직 준비되지 않았어요."))
```

get()은 딕셔너리에서 값을 꺼내고, 없을 때 대신 보여줄 문장도 함께 넣을 수 있습니다.

get()

한 줄 응답

미니 봇

딕셔너리에서 자주 쓰는 내장 함수

찾아보고 사용해보고 출력해보세요

`get(이름표)`, `keys()`, `values()`, `items()`, `update()`, `pop(이름표)`

```
1 student = {  
2     "이름": "알렉스",  
3     "나이": 20,  
4 }  
5 student.get("이름")  
6
```

`get()` 딕셔너리에서 값 꺼내기

팀별 활동

딕셔너리 {}

3개 만들어보기!

```
1 student = {  
2     "name": "지민",  
3     "grade": 1  
4 }  
5  
6 account = {  
7     "bank": "KB",  
8     "money": 5000  
9 }
```

"bank" 항목 이름

5000 그 항목의 값

{ } 딕셔너리 표시

import json?!

방금 만든거 json 모듈을 불러서 만들어 볼 것!

왜 json 형식이 딕셔너리 보다 실무에서 더 좋을 수 있을까요??

```
import json

data = {"이름": "지용", "나이": 20}

# 딕셔너리 → JSON 문자열로 변환
json_data = json.dumps(data)
print(json_data)
```

오류는 왜 생길까

고칠 지점을 보여주는 신호

오류는 망했다는 뜻이 아니라, 어디를 고치면 되는지 알려주는 힌트입니다.

특히 입력값은 "문자냐 숫자냐"를 먼저 의심해보면 많이 해결됩니다.

```
1 age = input("나이: ")
2 print(age + 1)
3
4 # 고친 버전
5 age = int(input("나이: "))
6 print(age + 1)
```

input으로 받은 값은 기본적으로 문자이기 때문에 숫자 연산을 하려면 int()가 자주 필요합니다.

오류

문자/숫자

힌트

try / except

실수해도 프로그램이 멈추지 않게 하기

사용자가 잘못 입력했을 때 친절한 안내를 보여줄 수 있습니다.

실전에서는 오류를 막는 것만큼, 어떻게 안내할지도 중요합니다.

```
1 try:
2     age = int(input("나이: "))
3     print(age)
4 except ValueError:
5     print("숫자로 입력해주세요.")
6
7 try:
8     money = int(input("금액 입력: "))
9     print("입력한 금액:", money)
10 except ValueError:
11     print("숫자만 입력해주세요.")
```

이번 자료에서는 가장 자주 만나는 ValueError만 먼저 잡아도 충분합니다.

try

except

친절한 안내

쉬는 시간

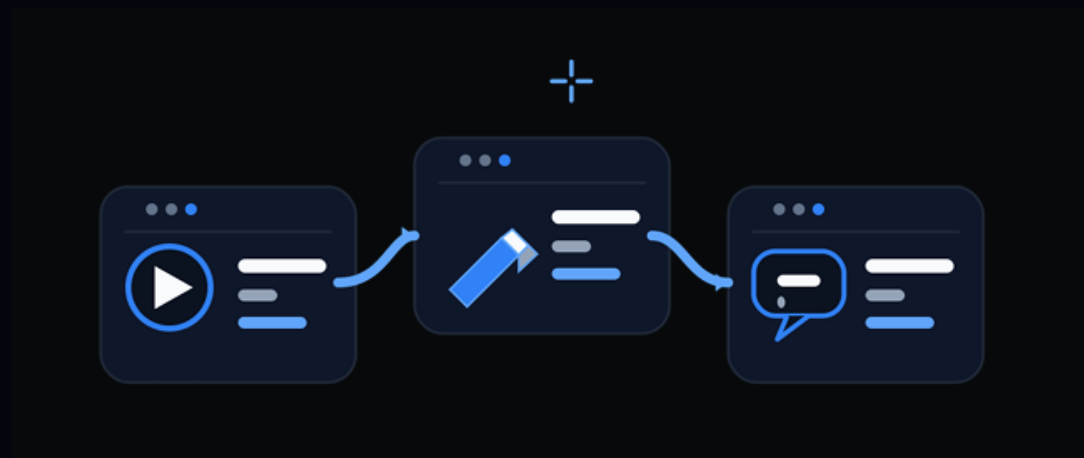
10분 휴식 + 소통 시간

함수가 많아지면 생기는 고민

비슷한 기능을 한 번에 정리할 수 없을까

- 여러 함수를 한 번에 다루는 방법은 없을까?
- 비슷한 기능을 하나의 틀로 묶을 수 없을까?

이 다음 단계에서 “클래스”라는 개념이 등장합니다.



함수 복습하기

기능 하나부터 차근차근 이해하기

클래스로 가기 전, 함수가 무엇을 해주는지 다시 한 번 확인합니다.

```
1 def introduce(name):  
2     print("안녕하세요, 저는", name, "입니다  
3     .")  
4 introduce("홍길동")  
5  
6 def bank_bot(name):  
7     print("안녕하세요, 저는", name, "상담봇입  
8     니다.")  
9 bank_bot("예금")
```

여기까지 확실히 이해되면 오늘 핵심은 이미 충분히 잡은 것입니다.

함수 복습

이름 넣기

출력

그다음 단계

기본값 넣기

값을 안 넣어도 기본으로 동작하게 만들 수 있습니다.
이 기능은 함수 사용을 조금 더 편하게 해줍니다.

```
1 def introduce(name="홍길동"):  
2     print("안녕하세요, 저는", name, "입니다  
3     .")  
4 introduce()  
5 introduce("김지민")
```

name="홍길동"은 값을 안 넣으면 홍길동을 기본으로 쓴다는 뜻
입니다.

참고

바깥 변수 사용하기

함수 밖에 있는 값을 함수 안에서 가져와 쓸 수도 있습니다.

```
1 name = "홍길동"
2
3 def introduce():
4     print("안녕하세요, 저는", name, "입니다")
5
6 introduce()
```

오늘은 "함수 안에서도 바깥 값을 가져올 수 있다" 정도만 기억해도 충분합니다.

[참고](#)[바깥 변수](#)[흐름만 보기](#)

클래스: 오늘의주인공

클래스: 값과 기능을 함께 묶는 것

앞에서는 이름과 기능을 따로 생각했습니다.

- 이제는 값과 기능을 하나의 설계도로 함께 묶어봅니다.

```
1 class StudentBot:
2     def __init__(self, name):
3         self.name = name
4
5     def introduce(self):
6         print("안녕하세요,", self.name, "입니다.")
```

class는 설계도, self.name은 그 설계도로 만든 대상이 가진 값이라고 이해하면 됩니다.

클래스 예시

이름을 가진 비서를 만드는 틀

함수 여러 개와 관련 값을 함께 모아둘 수 있다는 점이 클래스의 핵심입니다.

```
1 class SavingBot:
2     def __init__(self, name, product):
3         self.name = name
4         self.product = product
5
6     def introduce(self):
7         print("안녕하세요,", self.name, "입
8             니다.")
9
10    def recommend(self):
11        print(self.product, "설명을 시작합니
12            다.")
```

이번에는 name과 product라는 두 값을 함께 들고 다니는 틀이라고 보면 됩니다.

class 설계도
만들기

__init__ 처음 만들 때
실행

self.name = name
받은 이름 저장

introduce
자기소개 기능

클래스로 만든 비서 사용하기

설계도로 실제 비서 만들기

설계도만 있는 것이 아니라,
실제 도우미를 만들어 사용합니다.

```
1 bot1 = SavingBot("금융 비서", "예금")
2 bot1.introduce()
3 bot1.recommend()
4
5 bot2 = SavingBot("절약 비서", "적금")
6 bot2.introduce()
```

bot1, bot2는 설계도로 실제 만든 대상입니다.

객체

실행

미리보기

미니 에이전트 만들어보기

입력받고 → 규칙 확인하고 → 답하는 프로그램

처음 단계의 에이전트는 생각보다 단순합니다.
질문을 받고, 미리 정한 규칙에 따라 답하면 됩니다.



가장 쉬운 미니 에이전트

클래스 없이도 만들 수 있습니다.

딕셔너리와 입력만으로도 충분히 “답변하는 프로그램” 느낌을 만들 수 있습니다.

```
1 answers = {  
2     "공부": "오늘 할 일을 10분짜리로 쪼개보세요.",  
3     "진로": "좋아하는 것과 잘하는 것을 같이 적어보세요.",  
4     "친구": "상대 이야기를 먼저 들어보세요.",  
5     "저축": "작게라도 목표 금액을 정해보세요."  
6 }  
7  
8 topic = input("고민을 입력하세요: ")  
9 print(answers.get(topic, "조금 더 자세히 말해  
    줄래요?"))
```

입력받고, 준비한 규칙을 확인하고, 맞는 문장을 답하는 흐름입니다.

클래스 버전 미니 에이전트

도우미 안에 답변 기능을 넣기

```
1 class StudentAgent:
2     def __init__(self):
3         self.answers = {
4             "공부": "오늘 할 일을 10분짜리로 나눠보세요.",
5             "진로": "좋아하는 것과 잘하는 것을 같이 적어보세요."
6         }
7
8     def respond(self, topic):
9         return self.answers.get(topic, "조금 더 자세히 말해줄래요?")
10
11 agent = StudentAgent()
12 question = input("고민: ")
13 print(agent.respond(question))
```

핵심은 "딕셔너리 응답 기능을 클래스 안으로 넣을 수도 있다"는 점입니다.

나는 오늘

코드를 조금 덜 무서워하게 됐다.

모든 문법을 다 기억하지 못해도 괜찮습니다.
작은 기능 하나를 직접 만들어본 감각이 남으면 충분합니다.



팀 대결

배운 내용을 짧고 빠르게 다시 써보는 시간

- 1라운드: 기호/명령어 뜻 맞히기
- 2라운드: 출력 결과 예측하기
- 3라운드: 오류 1개 고치기
- 4라운드: 빈칸 채워 코드 완성하기
- 5라운드: 팀별 미니 금융 프로그램 제작



마지막 라운드는 "실제로 돌아가는 작은 프로그램"이면 충분합니다.

퀴즈

오류 수정

미니 프로그램